

Fireblocks Flow

12 min read

Fireblocks Flow lets you accept crypto deposits, send withdrawals, and convert assets across chains and tokens through a single configurable flow.

Built in partnership with Dynamic, a Fireblocks company, Flow handles the routing, conversion, and compliance between a user's funding source and your settlement destination, which can be a Fireblocks vault, your own self-custody address, an embedded wallet, or an external wallet address. The same primitive supports user-initiated deposits, treasury withdrawals to user wallets, and cross-chain or cross-token conversions; you configure one checkout and choose which flow each transaction represents. Fireblocks Flow is currently in Early Access.

How it works

Flow centers on a single object called a *checkout*: a server-side configuration that defines where funds settle (the token, chain, and address) and which mode the checkout operates in. Once a checkout exists, your application creates a checkout transaction against it whenever a user wants to deposit, withdraw, or convert.

A typical transaction moves through five stages:

1. Your application creates a checkout transaction with an amount and currency, plus optional destination overrides.
2. The user attaches a source: a self-custody wallet, an exchange account, or another address.
3. The platform returns a quote based on the selected token that locks in the conversion route and rate.
4. The user signs the transaction in their wallet, and the platform broadcasts it to the source chain.
5. Funds route through any required bridging or swapping and settle at the destination.

HMAC-signed webhooks fire at every state transition so your backend can track progress without polling.

Two integration paths are available. The Dynamic JavaScript SDK is the higher-level option for web and mobile applications with a user-facing UI; one SDK call typically wraps several underlying REST calls and handles signing through the user's connected wallet. The Dynamic REST API is the lower-level option for server-to-server use, agentic flows, and bespoke clients where you need explicit control over each step. A pre-built UI widget is on the roadmap but not yet available. During Early Access, Flow is API-first.

For implementation details, see Dynamic's [Checkout Flow end-to-end guide](https://www.dynamic.xyz/docs/javascript/reference/client/checkout-flow)

(<https://www.dynamic.xyz/docs/javascript/reference/client/checkout-flow>), and the [Deposit with Crypto product overview](https://www.dynamic.xyz/docs/overview/deposit-with-crypto) (<https://www.dynamic.xyz/docs/overview/deposit-with-crypto>).

Supported sources and destinations

The Dynamic JavaScript SDK supports the following source chains on mainnet:

- BTC
- EVM (Ethereum, Polygon, Arbitrum, Base, and other EVM-compatible chains)
- Solana
- Sui

Users can also fund a checkout from supported exchange accounts:

- Coinbase
- Kraken
- Crypto.com
- Binance

Funds settle to any on-chain address you provide. This can be a Fireblocks vault deposit address, an address your application controls directly, a Dynamic embedded wallet, or an external wallet address. The destination is set per chain on the checkout (unless overridden), with one entry for each settlement chain you support.

Coming soon

The following are on the Flow roadmap:

- A pre-built UI widget for embedding the checkout experience without building your own interface
- Per-transaction deposit addresses as a funding source, letting users (or systems) send funds without connecting a wallet or exchange
- Robinhood as a supported exchange source

Configuring a checkout

A checkout defines two things: the settlement target (which token, on which chain, to which address) and the operating mode. Most integrators have one or a small number of checkouts (e.g., one per product surface or settlement currency).

Flow has two operating modes:

- **Payment mode:** You set the amount the user pays. Use this for fixed-price checkouts where the merchant or platform determines the total (e.g., a \$25 USDC purchase).
- **Deposit mode:** The user sets the amount. Use this for account funding flows where the user decides how much to add.

A checkout's destination is configured per chain. If you want to settle on multiple chains, provide one destination entry per chain, each with its own address.

You can update a checkout's destination address after creation through the Dynamic REST API. The new address applies only to transactions created after the update; any in-flight transaction continues settling to the destination set when it was created.

Using a Fireblocks vault

A Fireblocks vault can act as either the destination or the source for a checkout transaction. Fireblocks Flow doesn't distinguish between a vault deposit address and any other on-chain address (both are just identifiers), but how you use them depends on the flow.

Vault as a settlement destination

For deposits, generate a deposit address on the destination vault for the settlement asset and chain (for example, USDC on Ethereum), then use that address as the destination on the checkout. Settled funds arrive at the vault on the configured chain. To get a deposit address, open the vault in the Fireblocks Console at [Accounts > Vault](https://console.fireblocks.io/v2/accounts/vault) (<https://console.fireblocks.io/v2/accounts/vault>).

Vault as a source for withdrawals

For withdrawals from a Fireblocks-managed treasury to user wallets, configure the checkout with the vault as the source-side address and override the destination per transaction with the user's wallet address. This lets a single checkout configuration handle every user. For example, you can withdraw SOL from a vault and deliver USDC on Ethereum to the user's self-custody address in one flow.

Inbound deposits arrive from external addresses

Deposits settled into a Fireblocks vault through Flow arrive from external, non-whitelisted addresses by design. The sender can be the end-user's own wallet, or an intermediate bridge or DEX contract used during settlement routing. This is the expected behavior, not a security event.

How you handle these inbound transfers depends on your vault's Policy rules. You can either whitelist senders as they appear, or operate the vault with a Policy rule that permits inbound transfers from unwhitelisted external addresses. For configuration guidance, see [About Policies](https://support.fireblocks.io/hc/en-us/articles/19156998685980-About-Policies) (<https://support.fireblocks.io/hc/en-us/articles/19156998685980-About-Policies>).

Transaction states

Every checkout transaction has three independent state axes. When you're inspecting a transaction or troubleshooting an issue, pull all three.

Execution state

Tracks how the transaction is moving through the payment lifecycle, from creation to source-chain confirmation:

State	Meaning
initiated	Transaction created; no source attached yet.
source_attached	User linked a wallet or exchange account.
quoted	A swap or bridge quote is locked. Quotes have a short time-to-live; if the user doesn't sign promptly, the quote expires and must be refreshed.
signing	Awaiting the user's signature in their wallet.
broadcasted	Signed transaction submitted to the source chain.
source_confirmed	Source chain has confirmed the transaction.
cancelled	Cancelled before broadcast.
expired	Quote or session expired without progressing.
failed	Unrecoverable failure: check the failure code.

Settlement state

Tracks the post-broadcast flow of funds through bridging, swapping, and final delivery:

State	Meaning
none	Default before broadcast.
routing	Selecting the settlement path.
bridging	Cross-chain bridge in progress.
swapping	DEX swap on the destination chain.
settling	Final settlement transfer.
completed	Funds delivered to the destination. Terminal success.
failed	Settlement pipeline failed: check the failure code.

Risk state

Tracks the outcome of compliance screening:

State	Meaning
unknown	Initial.
pending	Screening in progress.
cleared	Passed screening.
blocked	Hard block from sanctions or geographic restrictions.
review	Flagged for manual review.

A common point of confusion is the difference between `source_confirmed` and `completed`. The first means the user's transaction has the required confirmations on the source chain. The user has done their part. The second means the funds have arrived at the destination after any routing, bridging, or swapping. Source confirmation always happens first; settlement follows.

Webhooks

Webhooks are the recommended way for your backend to learn about transaction state changes. Three event types fire for checkout transactions:

- `checkout.transaction.execution.updated`: execution state changed
- `checkout.transaction.settlement.updated`: settlement state changed
- `checkout.transaction.risk.updated`: risk state changed

Each event carries a state-transition payload, not the full transaction. The payload includes the axis that changed, the previous and new states, the `transactionId`, a timestamp, and any axis-specific additional data. To work with the full transaction record, call `getCheckoutTransaction` with the `transactionId` from the event.

Delivery follows standard best practices:

Property	Behavior
Retries	Up to 5 on any non-2XX response
Backoff	15 seconds → 1 minute → 10 minutes → 1 hour → 1 day
Endpoint timeout	15 seconds for a 2XX response
Ordering	Not guaranteed; events can arrive out of order
Deduplication	Use the <code>messageId</code> field
HTTPS	Required; HTTP endpoints are refused

Webhooks auto-disable on repeated failure. In live environments, a webhook is disabled after 6 failed attempts on a single message or 1,500 failed messages over 30 days. In sandbox environments, the thresholds are 3 per message and 250 per 30 days. Successful retries don't count against the failure threshold. A disabled webhook stays disabled until manually re-enabled in the Dynamic dashboard. This is the most common cause of "we stopped receiving events" issues; check the webhook status first.

Each webhook signs every payload using HMAC-SHA256 with a per-webhook signing secret. Verify signatures against the `x-dynamic-signature-256` header using constant-time comparison. For setup steps and a verification code sample, see Dynamic's [webhook setup documentation](https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/setup) (<https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/setup>) and the [event delivery best practices guide](https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/delivery-best-practices) (<https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/delivery-best-practices>).

Verification and compliance

Two verification layers run on every transaction by default:

- **Sanctions screening** runs against the source address through Chainalysis. A hit surfaces as a `SANCTIONS_BLOCK` failure and sets the risk state to `blocked`.
- **Geographic restrictions** block traffic from sanctioned regions based on the user's IP address. A hit surfaces as a `GE0_BLOCK` failure and sets the risk state to `blocked`.

Three additional verification features are available as optional configuration:

- **Spam token filtering** blocks known spam tokens from being accepted as a source asset. Available when wallet signature verification is enabled.
- **Name matching** verifies the depositor's identity against an expected name on the transaction.
- **Wallet signature verification** requires the user to sign a message at connect time to prove ownership of the source wallet. Uses Dynamic's Connect & Sign flow in the JavaScript SDK.

If you need verification beyond the default Chainalysis coverage, you'll need to layer your own checks on top. There's no path to bring a different screening provider through Dynamic at this time.

Reconciling transactions

Each checkout transaction can carry a memo: an arbitrary string you attach when the transaction is created. The memo persists through the entire transaction lifecycle and is included in every webhook payload, which makes it the recommended way to tie a Flow transaction to your own order ID, invoice number, user identifier, or other internal reconciliation key.

Memos also satisfy chain-level requirements for networks like Stellar and Cosmos where a memo or tag is required to route funds correctly to the recipient.

On the Fireblocks side, the memo travels with the transaction and is the cleanest way to match an inbound vault deposit back to the originating Flow transaction.

Troubleshooting

A transaction is stuck in signing

The user opened the signing prompt but didn't complete it before the quote's time-to-live elapsed. The transaction's failure code will be QUOTE_EXPIRED. Have the user start a new transaction; a fresh quote will be issued.

Funds never arrived at the destination

Pull the transaction and check the settlement state. If it shows completed, the funds are at the destination. Confirm the destination address and check the vault. If it's stuck in bridging, swapping, or settling, look for a BRIDGE_FAILED or DESTINATION_TIMEOUT failure code. Both indicate a downstream routing issue that requires engineering investigation.

Webhooks stopped arriving

Check whether the webhook has been auto-disabled in the Dynamic dashboard. Repeated non-2XX responses from your endpoint will disable it; you'll need to re-enable it manually. Also verify your endpoint is returning 2XX within the 15-second timeout for all event types, not just some.

Signature verification on incoming webhooks fails

This usually comes down to one of three things: hashing the wrong byte representation of the payload, using the wrong signing secret (the per-webhook secret, not your environment API token), or a recently rotated secret that wasn't updated on the receiving end. For step-by-step verification guidance, see Dynamic's [webhook setup documentation](https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/setup) (<https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/setup>).

The API returns 403 Forbidden

Risk screening blocked the transaction. Check the transaction's risk state. A value of blocked indicates a sanctions or geographic block. See the verification and compliance section for what these blocks mean.

The API returns 409 Conflict on broadcast

The client retried after a partial success and submitted the same transaction hash twice. The first submission is authoritative; no action is needed.

The API returns 422 with no quote available

No route exists between the source token or chain and the settlement target. Confirm the pair is in the supported automatic conversion list: EVM, Solana, Sui, and Bitcoin.

A deposit landed in a Fireblocks vault from an address that isn't whitelisted

Expected behavior. Deposits routed through Flow can arrive from the user's own wallet or from a bridge contract used during settlement. See the inbound deposits note in the "Using a Fireblocks

vault" section.

Failure codes

When a transaction ends in a failed state, a failure code on the transaction object identifies the cause. Codes fall into three groups by who can resolve them.

Customer self-resolvable

The user can retry or correct the issue themselves:

Code	What happened
QUOTE_EXPIRED	The quote's time-to-live elapsed before the user signed. Refresh the quote and retry.
USER_REJECTED	The user rejected the signing prompt in their wallet.
INSUFFICIENT_FUNDS	The source balance was below the required amount.
SLIPPAGE_EXCEEDED	The executed price moved past the slippage tolerance during the swap. Retry; if it persists on a major trading pair, contact support.
TX_REVERTED	The on-chain transaction reverted, often from a missing token allowance or insufficient gas.

Configuration or policy

The merchant or platform may need to adjust configuration:

Code	What happened
GEO_BLOCK	The user's IP address is in a restricted region.
SANCTIONS_BLOCK	Sanctions screening flagged the source address.
TOKEN_BLOCKED	The token pair isn't in the configured allowlist. Adjust the checkout's settlement or restriction configuration.
REFUND_INITIATED	A refund was issued back to the source after a downstream leg failed. Informational; set automatically as a recovery action.

Engineering escalation

Contact Fireblocks Support with the `transactionId`:

Code	What happened
SUBMISSION_FAILED	The signed transaction never landed on chain. Usually transient.
MAX_POLLING_RETRIES	Source-chain confirmation polling exhausted its retries. Often an RPC or chain-health issue.
BRIDGE_FAILED	The cross-chain bridge leg failed.
DESTINATION_TIMEOUT	Settlement exceeded the maximum allowed time.
INTERNAL_ERROR	An unhandled server error.

When you contact support, include the `transactionId`, the environment (live or sandbox), the failure code, the values of all three state axes, and the approximate time of the failure.

For HTTP-level errors: 400 typically means a missing required field (often `fromChainName`, `fromChainId`, `fromAddress`, or `txHash`); 403 indicates the transaction was blocked by risk screening; 404 means the checkout or transaction ID wasn't found (verify the environment ID); 409 indicates a state conflict or duplicate submission; 422 indicates a validation error or no available quote.

Disclaimers

Your application should make clear to end-users that asset conversion and cross-chain routing are executed by independent third-party providers. Users keep full control of their assets and must explicitly sign each transfer. On-chain transactions are final and cannot be reversed.

Dynamic does not control the swap, bridge, or routing protocols used to convert and deliver assets. Rates and fees are sourced from third-party providers and may change between quote and execution. Cross-chain transfers carry risk, including slippage, partial fills, and failed conversions. On-chain transactions are final and cannot be reversed. These materials are not investment, financial, legal, or tax advice. You are responsible for evaluation at your own discretion. Fireblocks Flow is infrastructure. Please review Dynamic terms and conditions for full details on acceptable use.

Related resources

- [Deposit with Crypto product overview](https://www.dynamic.xyz/docs/overview/deposit-with-crypto) (<https://www.dynamic.xyz/docs/overview/deposit-with-crypto>): the Dynamic-side product page
- [Checkout Flow end-to-end guide](https://www.dynamic.xyz/docs/javascript/reference/client/checkout-flow) (<https://www.dynamic.xyz/docs/javascript/reference/client/checkout-flow>): implementation reference for the JavaScript SDK
- [Setting up webhooks](https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/setup) (<https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/setup>): webhook registration and signature verification
- [Webhook event reference](https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/events) (<https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/events>): full payload structure and event types

- [Event delivery best practices](https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/delivery-best-practices) (<https://www.dynamic.xyz/docs/overview/developer-dashboard/webhooks/delivery-best-practices>): retries, timeouts, and idempotency

Related articles

-  Authentication and authorization
-  Fireblocks Network Link for providers
-  Interacting with the TRUST platform
-  On/Off-ramp orders overview
-  Welcome to Fireblocks
-  Solana dApps interactions Using Program Calls
-  About Travel Rule transaction screening
-  EDX Markets Account